UNIVERSIDADE DE VASSOURAS
CAMPUS SAQUAREMA

# ROTEIRO DE APRESENTAÇÃO

## Migração de algoritmos em C, Java e COBOL para Python

Oito apresentadores • 15 a 17 minutos • 10 slides sugeridos

Material de apoio: códigos simplificados em Python, documentação de requisitos, diagramas UML, modelos SQL e testes automatizados.

Observação: os nomes são os integrantes do Projeto SIREN. O tema desta apresentação é a atividade independente de migração de algoritmos.

# Organização da apresentação

A fala foi distribuída para que cada integrante explique um aspecto efetivamente presente nos arquivos entregues, sem apresentar os bancos de dados como funcionalidades já implementadas.

| Apresentador | Slide(s) | Tema | Tempo |
|---|---|---|---|
| Thiago Pereira Santiago | 1 | Abertura e objetivos | 1min30 |
| Eduardo Antony Aguiar Guimarães | 2 | Migração e equivalências de linguagens | 1min45 |
| Jenifer Gusmão Barbosa | 3–4 | Requisitos, casos de uso e UML | 2min |
| Juan Frota de Oliveira | 5 | Funcionários: C → Python | 2min |
| Juan Rodrigues Laureano de Souza | 6 | Alunos: Java → Python | 2min |
| Mateus Nascimento Sant'anna Luz | 7 | Média: COBOL → Python | 2min |
| Thiago Almeida das Chagas | 8 | Votação: Java → Python | 2min |
| Wesley Abreu de Aguiar | 9–10 | Bancos de dados, testes e conclusão | 2min |

## Sequência sugerida de slides

1. Capa: título, instituição e integrantes
2. O que foi migrado e como os programas são executados
3. O que são requisitos funcionais e não funcionais
4. Casos de uso, diagramas UML e proposta de tabelas
5. Funcionários: operações e classes
6. Alunos em Java: dicionário e lista
7. COBOL: cinco alunos e média aritmética
8. Sistema de votação: três opções e resultados
9. Entidades, tabelas e bancos de dados possíveis
10. Testes, limitações e conclusão

## 1. Thiago Pereira Santiago

Tempo estimado: 1min30 | Slide 1 — Capa, objetivos e atividade

Apoio visual: Mostrar a capa e os cinco itens do enunciado: migração, requisitos, UML, entidades e bancos de dados.

### Fala sugerida

Boa tarde. Nosso trabalho é sobre migração de algoritmos para Python. Recebemos quatro programas escritos em linguagens diferentes: um em C, dois em Java e um em COBOL. A proposta não era criar sistemas completamente novos, mas reescrever esses algoritmos em Python, preservando as funções principais de cada um.

Além da programação, a atividade pediu documentação de requisitos, diagramas de classes, identificação das possíveis entidades e tabelas e indicação dos bancos de dados que poderiam ser usados. Portanto, nosso foco foi entender tanto o funcionamento dos códigos quanto a maneira de documentar e organizar um sistema.

Os quatro exemplos são um cadastro de funcionários, um cadastro de alunos e notas, o cálculo da média de cinco alunos e um sistema de votação. Organizamos os arquivos por programa, com códigos, documentação, diagramas, propostas de SQL e testes. Agora vamos mostrar como essa migração foi feita e como verificamos os resultados.

**Transição: Eduardo vai explicar quais estruturas das linguagens originais foram utilizadas e como as representamos no Python.**

## 2. Eduardo Antony Aguiar Guimarães

Tempo estimado: 1min45 | Slide 2 — Estratégia e comparação entre linguagens

Apoio visual: Tabela: C/struct+vetor → classe+lista; Java/HashMap+ArrayList → dict+list; COBOL/OCCURS → lista+for; Java/arrays+while → listas+while. Mostrar arquivo executar.py.

### Fala sugerida

Nosso primeiro passo foi observar as entradas, as saídas e as regras de cada código. Depois procuramos estruturas equivalentes em Python. Por exemplo, no programa em C, os dados do funcionário estavam organizados em uma struct e em um vetor. Em Python usamos uma classe para o funcionário e uma lista para guardar os registros.

No programa de alunos em Java, existiam um HashMap para as notas e um ArrayList para os nomes. Na versão em Python, essas estruturas viraram um dicionário e uma lista. No COBOL, os cinco registros eram controlados por posições fixas e comandos de repetição. Em Python usamos um for e objetos das classes Aluno e Turma. Já o sistema de votação manteve as listas de opções e contadores e a repetição até o usuário decidir encerrar.

Também criamos um arquivo executar.py, que abre um menu para executar os quatro programas e os testes. Ele identifica a pasta em que o próprio arquivo está, então podemos extrair o projeto em Downloads ou na Área de Trabalho e executá-lo com Python 3, sem precisar de arquivo .bat.

Um ponto importante: os programas continuam simples e funcionam no terminal; seus dados permanecem apenas enquanto estão abertos.

**Transição: Jenifer vai apresentar como registramos esses comportamentos na documentação de requisitos.**

## 3. Jenifer Gusmão Barbosa

Tempo estimado: 2min | Slides 3 e 4 — Requisitos, casos de uso, diagramas e modelagem

Apoio visual: Exibir trechos das tabelas RF/NF e um diagrama UML: Funcionario e SistemaFuncionarios. Indicar que o PDF da UFJF inspirou a organização, não a funcionalidade.

### Fala sugerida

A segunda parte do trabalho foi a documentação. Utilizamos como referência estrutural o Documento de Requisitos do Sistema, da Universidade Federal de Juiz de Fora. A partir desse modelo, descrevemos cada programa separadamente, com introdução, requisitos, casos de uso, diagramas UML e possibilidades de banco de dados.

Os requisitos funcionais, identificados por RF, mostram o que o sistema precisa fazer. No programa de funcionários, por exemplo, RF001 é cadastrar o funcionário e RF004 é respeitar o limite de cem registros. Já os requisitos não funcionais, identificados por NF, descrevem características de execução, como funcionar no terminal com Python 3 e tratar informações numéricas inválidas.

Nos casos de uso, identificamos o usuário que realiza a ação, a condição necessária, os passos esperados e o que acontece quando surge um erro. Um exemplo é a busca por funcionário: o usuário informa um nome, o programa percorre a lista e apresenta o resultado ou informa que não encontrou.

Os diagramas de classes servem para visualizar a estrutura do código. Na UML, os atributos representam os dados guardados e os métodos representam as ações realizadas. No diagrama de funcionários, a classe Funcionario guarda nome, cargo e salário; já SistemaFuncionarios controla o cadastro, a listagem e a busca.

É importante separar isso do banco de dados: uma classe no Python não significa, obrigatoriamente, uma tabela real. As tabelas do nosso trabalho são propostas de como armazenar esses dados futuramente.

**Transição: Juan Frota vai apresentar o primeiro sistema, o cadastro de funcionários.**

# 4. Juan Frota de Oliveira

Tempo estimado: 2min | Slide 5 — Programa 1: funcionários (C → Python)

Apoio visual: Mostrar arquivo 01_funcionarios.py, diagrama Funcionario/SistemaFuncionarios e tabela proposta funcionarios(id, nome, cargo, salario).

## Fala sugerida

O primeiro algoritmo é o cadastro de funcionários, originalmente escrito em C. Ele permite adicionar, listar e buscar funcionários pelo nome. O sistema também possui um limite de cem registros, regra que mantivemos na versão Python.

Para organizar o código, usamos duas classes. A primeira é Funcionario, que guarda nome, cargo e salário. A segunda é SistemaFuncionarios, que possui a lista de funcionários e os métodos adicionar_funcionario(), listar_funcionarios() e buscar_funcionario(). Quando cadastramos alguém, o método verifica se os campos foram informados, se o salário não é negativo e se ainda há espaço na lista.

Na demonstração, podemos selecionar a opção 1, cadastrar Ana como Analista com salário de 3200 reais, e depois utilizar a opção 2 para listar o cadastro. Na opção 3, pesquisamos Ana e recebemos o cargo e o salário. Se procurarmos um nome que não está cadastrado, o programa informa que ele não foi encontrado.

No diagrama UML, mostramos que SistemaFuncionarios reúne de zero a cem funcionários. Para um banco de dados futuro, sugerimos a tabela funcionarios, com um identificador, nome, cargo e salário. Atualmente essa tabela é apenas um modelo, porque os registros ainda ficam em memória.

**Transição: Juan Rodrigues vai explicar a migração do programa de alunos, originalmente feito em Java.**

## 5. Juan Rodrigues Laureano de Souza

Tempo estimado: 2min | Slide 6 — Programa 2: alunos e notas (Java → Python)

Apoio visual: Mostrar arquivo 02_alunos_java.py e o diagrama de classe SistemaAlunos; enfatizar dict notas e list lista_nomes.

### Fala sugerida

O segundo programa veio de Java e faz o cadastro de alunos e suas notas finais. Suas opções são adicionar aluno, listar alunos, buscar uma nota pelo nome e sair. Para manter a ideia original, utilizamos a classe SistemaAlunos com duas estruturas: o dicionário notas e a lista lista_nomes.

O dicionário associa cada nome a uma nota, enquanto a lista mantém a ordem dos nomes cadastrados. Os principais métodos são adicionar_aluno(), listar_alunos() e buscar_nota(). Por exemplo, ao cadastrar Beatriz com nota 8,5, podemos consultar o nome e o sistema mostra a nota correspondente.

Uma particularidade importante do código original foi preservada. Se cadastrarmos o mesmo nome duas vezes, ele aparece duas vezes na lista, mas a nota consultada passa a ser a última armazenada no dicionário. Isso ocorre porque o nome foi utilizado como chave. Não significa que o sistema consiga distinguir dois alunos diferentes que tenham o mesmo nome.

No diagrama, temos apenas a classe SistemaAlunos, porque é essa a classe que existe no Python. Como melhoria para uma futura versão com banco de dados, propusemos uma tabela alunos com id, nome e nota_final. O id permitiria distinguir os registros sem depender apenas do nome.

**Transição: Mateus vai explicar a conversão do exemplo em COBOL, que calcula a média de uma turma.**

## 6. Mateus Nascimento Sant’anna Luz

Tempo estimado: 2min | Slide 7 — Programa 3: cinco alunos e média (COBOL → Python)

Apoio visual: Mostrar arquivo 03_alunos_cobol.py e diagrama Aluno/Turma; apresentar a conta (8 + 6 + 7 + 9 + 10) ÷ 5 = 8,00.

### Fala sugerida

O terceiro código foi escrito originalmente em COBOL. Diferentemente dos outros programas, ele não usa um menu. A execução pede exatamente cinco nomes e cinco notas, mostra os registros e calcula a média aritmética da turma.

Na versão em Python, criamos a classe Aluno, que guarda nome e nota, e a classe Turma, que mantém uma lista de alunos e controla a quantidade máxima de cinco. O método cadastrar_aluno() valida os dados e o método calcular_media() soma as notas e divide pela quantidade cadastrada.

Para demonstrar, podemos registrar as notas 8, 6, 7, 9 e 10. A soma é 40 e a divisão por cinco resulta na média 8,00. O programa também apresenta as notas com duas casas decimais.

Mantivemos a faixa de notas entre zero e 99,99 porque esse era o formato permitido pelo campo numérico do COBOL original, não porque essa seja a escala comum de avaliação acadêmica. A migração também evita uma limitação do campo usado para guardar a soma no código de origem.

O diagrama mostra que uma Turma reúne até cinco objetos Aluno. Para um possível banco, sugerimos as tabelas turmas e alunos_turma, relacionadas pelo campo turma_id. Essa divisão permitiria armazenar diferentes turmas em uma evolução do sistema.

**Transição: Thiago Almeida vai apresentar a migração do sistema de votação.**

## 7. Thiago Almeida das Chagas

Tempo estimado: 2min | Slide 8 — Programa 4: votação (Java → Python)

Apoio visual: Mostrar arquivo 04_votacao.py, classe SistemaVotacao, vetor de contagem [1, 0, 2] e tabela proposta opcoes_votacao.

### Fala sugerida

O último algoritmo foi originalmente desenvolvido em Java e apresenta um sistema de votação bastante simples. Existem três opções disponíveis. O usuário digita 1, 2 ou 3 para votar, e a opção zero encerra a votação e apresenta os resultados.

Em Python, criamos a classe SistemaVotacao, com uma lista que guarda os nomes das três opções e outra lista que guarda a quantidade de votos. O método registrar_voto() verifica se a escolha está entre um e três e aumenta o contador da opção correspondente. O método obter_resultados() devolve os totais para apresentação.

Por exemplo, se digitarmos 1, 3, 3 e depois zero, o resultado será um voto para a opção 1, nenhum para a opção 2 e dois votos para a opção 3. Caso seja digitada uma opção inválida, ela não é contabilizada.

O diagrama UML contém uma única classe, SistemaVotacao. Para o banco de dados futuro, a proposta é uma tabela opcoes_votacao com id, descrição e total de votos. Este não é um sistema eleitoral completo: não identifica eleitores nem impede que uma mesma pessoa vote mais de uma vez. Nosso objetivo foi reproduzir a lógica do algoritmo fornecido.

**Transição: Wesley vai finalizar com as propostas de armazenamento, os testes e as conclusões do trabalho.**

## 8. Wesley Abreu de Aguiar

Tempo estimado: 2min | Slides 9 e 10 — SQL, testes, limitações e encerramento

Apoio visual: Mostrar resumo das cinco tabelas: funcionarios, alunos, turmas, alunos_turma, opcoes_votacao. Depois, exibir o terminal com “Ran 12 tests” e “OK”.

### Fala sugerida

Além de migrar os programas, nós identificamos como seus dados poderiam ser organizados em um banco relacional. Ao todo, propusemos cinco tabelas: funcionarios, alunos, turmas, alunos_turma e opcoes_votacao. A parte de média usa duas tabelas porque existe uma relação entre a turma e os alunos.

Entre os bancos estudados, o SQLite é o mais simples para esses exemplos, porque funciona localmente em um arquivo. MySQL e PostgreSQL também seriam possibilidades para versões maiores ou com vários usuários. É importante destacar que os códigos entregues não utilizam um banco real para gravar os cadastros: os arquivos SQL são modelos de uma melhoria futura.

Para conferir a migração, preparamos testes automáticos de cadastro, busca, limites, cálculo de média, contagem dos votos, execução no terminal e criação das tabelas SQL de exemplo. Executamos o arquivo testar_programas.py e os 12 testes passaram.

Os resultados mostram que as funções principais dos quatro algoritmos foram mantidas. Ainda existem limitações, como a perda de dados ao encerrar os programas, o problema dos nomes repetidos no cadastro de alunos e a ausência de controle de eleitores na votação. Essas são oportunidades de melhoria, não funções que afirmamos ter implementado.

Concluindo, o trabalho mostrou que migrar algoritmos não é apenas trocar a sintaxe. Também envolve entender as regras, representar as classes, documentar os requisitos e verificar o comportamento por meio de testes. Obrigado pela atenção. Estamos à disposição para as perguntas.

# Guia rápido para a demonstração ao vivo

Antes da apresentação: extrair o ZIP inteiro, abrir a pasta Entrega_Migracao_Python no VS Code e confirmar que o Python 3 está instalado.

## Abrir o menu

No terminal da pasta principal: python executar.py (no Windows, se necessário, py executar.py). Selecionar de 1 a 4 para abrir um sistema ou 5 para os testes.

## Funcionários

Opção 1 no menu principal → cadastrar Ana, cargo Analista, salário 3200 → listar → buscar Ana → sair.

## Alunos Java

Opção 2 → cadastrar Beatriz, nota 8,5 → consultar Beatriz → sair.

## Média COBOL

Opção 3 → informar cinco nomes e as notas 8, 6, 7, 9 e 10 → confirmar média 8,00.

## Votação

Opção 4 → digitar 1, 3, 3 e 0 → confirmar totais de 1, 0 e 2 votos.

## Testes automáticos

Opção 5 ou executar python testes/testar_programas.py na pasta principal → verificar que os 12 testes terminam com OK.

## Perguntas prováveis e respostas

### Os programas utilizam banco de dados?

Não. Trabalham com dados em memória. Os arquivos SQL descrevem tabelas possíveis e foram incluídos para atender à etapa de modelagem.

### Por que classes em Python se o código original era C ou COBOL?

As classes foram uma forma simples de organizar atributos e operações na migração, além de permitir diagramas UML coerentes com o código implementado.

### Qual a diferença entre requisito funcional e não funcional?

O requisito funcional descreve o que o programa faz, como cadastrar ou buscar. O não funcional descreve características de execução, como rodar no terminal e lidar com uma entrada inválida.

### Como foi testado o código?

O arquivo de testes verifica operações e regras dos quatro programas, a execução no terminal e os arquivos SQL. Nesta versão, os 12 testes automáticos passaram.

### Por que a nota do COBOL pode chegar a 99,99?

A declaração PIC 99V99 do arquivo original permite dois dígitos inteiros e dois decimais. Isso foi preservado para respeitar o formato do algoritmo, não para definir a nota máxima de uma instituição.

### O que acontece com nomes repetidos no programa de alunos?

A lista registra o mesmo nome de novo, mas o dicionário guarda a nota mais recente para aquela chave. Um identificador único no banco seria uma melhoria futura.

### O programa de votação impede votos repetidos?

Não. Ele apenas conta votos por opção e não identifica votantes, seguindo o escopo do algoritmo recebido.

### Por que escolher SQLite em vez de MySQL ou PostgreSQL?

SQLite é simples para armazenar localmente dados desses exemplos. MySQL e PostgreSQL fazem mais sentido em sistemas de maior porte e com acesso compartilhado.

Base consultada: enunciado da atividade; quatro arquivos de origem; versão simplificada sem .bat; Documentação_Migracao_Algoritmos.pdf; modelos UML e SQL; testes/testar_programas.py.